



École Marocaine des Sciences de l'Ingénieur
Campus Casablanca

Guide de Référence pour l'Entraînement de Modèles de Deep Learning

Projet MLP – CNN – RNN/LSTM/GRU

Rapport de projet de fin d'année

Réalisé par : Ahmed Masbah

Filière : Ingénierie Informatique et Réseaux

Année : 4^e année (Cycle d'ingénieur)

Encadré par : Mme. HIDILA Zineb

Année universitaire 2025–2026

Table des matières

1	Introduction	3
1.1	Contexte et motivation	3
1.2	Contexte du projet à l'EMSI	4
1.3	Description générale du projet	4
1.4	Objectifs détaillés	5
1.5	Démarche méthodologique	5
1.6	Vue d'ensemble des architectures	6
2	Préparation des données	7
2.1	Exploration et compréhension des datasets	7
2.1.1	Dataset tabulaire – Breast Cancer Wisconsin	7
2.1.2	Dataset d'images – MNIST	8
2.1.3	Corpus textuel pour modèles séquentiels	9
2.2	Prétraitement	9
2.2.1	Données tabulaires (MLP)	9
2.2.2	Images (CNN)	10
2.2.3	Séquences textuelles (RNN/LSTM/GRU)	11
2.3	Séparation des données	12
3	Perceptron Multicouche (MLP)	13
3.1	Rappel théorique	13
3.2	Architecture proposée	13
3.3	Initialisation des paramètres	14
3.4	Résultats expérimentaux	15
3.5	Discussion	17
4	Réseau de Neurones Convolutif (CNN)	18
4.1	Rappel théorique	18
4.2	Architecture CNN pour MNIST	18
4.3	Stratégies d'entraînement	19
4.4	Résultats et courbes	20

5	Réseaux Récurrents : RNN, LSTM et GRU	22
5.1	Rappel théorique	22
5.2	Corpus textuel et tâche	22
5.3	Architectures et configuration	23
5.4	Stratégies d'entraînement	24
5.5	Résultats : pertes et perplexités	24
5.6	Validation qualitative par génération de texte	25
6	Processus d'entraînement et hyperparamètres	27
6.1	Fonctions de perte et optimiseurs	27
6.2	Hyperparamètres clés	28
7	Résultats et comparaisons	29
7.1	Performances sur les jeux de test	29
7.2	Visualisations obligatoires	30
8	Analyse critique et conclusion	31
8.1	Analyse critique	31
8.2	Conclusion	32

Chapitre 1

Introduction

1.1 Contexte et motivation

Au cours des dernières années, le *deep learning* s'est imposé comme une approche incontournable pour le traitement de données complexes dans des domaines variés : vision par ordinateur, traitement automatique du langage naturel, analyse de signaux, systèmes de recommandation, etc. Les architectures profondes permettent d'extraire automatiquement des représentations pertinentes à partir de données brutes, à condition de respecter une méthodologie rigoureuse en termes de préparation des données, choix de modèle et stratégies d'entraînement.

Dans le cadre du module « Deep Learning » de l'École Marocaine des Sciences de l'Ingénieur (EMSI), ce projet a pour objectif de mettre en pratique ces concepts théoriques à travers la réalisation d'un pipeline complet d'entraînement de modèles de deep learning sur trois types de données : *tabulaires*, *images* et *séquences textuelles*. L'idée directrice est de montrer que le choix de l'architecture doit être guidé par la nature des données et par le biais inductif associé à chaque modèle :

- un perceptron multicouche (MLP) traite des vecteurs de features numériques sans structure spatiale ou temporelle particulière ;
- un réseau convolutif (CNN) exploite explicitement la structure bidimensionnelle des images via des filtres locaux et le partage de poids ;
- un réseau récurrent (RNN, LSTM, GRU) modélise des dépendances temporelles en introduisant une mémoire d'état le long de la séquence.

1.2 Contexte du projet à l’EMSI

Ce travail s’inscrit dans le cursus de la filière *Ingénierie Informatique et Réseaux* à l’EMSI, en quatrième année du cycle d’ingénieur. Il constitue un projet de synthèse pour le module « Deep Learning », encadré par Mme. HIDILA Zineb, et vise à préparer l’étudiant à :

- concevoir des architectures adaptées à des problèmes réels ;
- implémenter ces architectures en utilisant des bibliothèques modernes telles que PyTorch ;
- analyser de manière critique les résultats obtenus et les limites des modèles.

Au-delà de l’aspect académique, le projet se veut proche des contraintes professionnelles : reproductibilité des expériences, rigueur dans la documentation, clarté de la structuration du code et capacité à expliquer les choix d’architecture et d’hyperparamètres lors d’une présentation orale.

1.3 Description générale du projet

Conformément au guide de référence fourni par l’encadrante, le projet couvre trois blocs principaux, chacun associé à un type de données et à une famille d’architectures :

- **Bloc tabulaire (MLP)** : mise en œuvre d’un perceptron multicouche pour la classification binaire sur le dataset *Breast Cancer Wisconsin*. Ce dataset, largement utilisé dans la littérature, permet de tester la capacité du MLP à exploiter des variables numériques structurées (rayon, texture, compacité, etc.) afin de distinguer des tumeurs bénignes de tumeurs malignes.
- **Bloc image (CNN)** : conception d’un réseau de neurones convolutif entraîné sur le dataset d’images manuscrites *MNIST*. Ce cas d’usage classique permet de mettre en évidence le biais inductif des CNN pour la reconnaissance de chiffres à partir d’images en niveaux de gris de taille 28×28 .
- **Bloc séquentiel (RNN / LSTM / GRU)** : développement de trois variantes de réseaux récurrents (RNN simple, LSTM et GRU) pour la modélisation de langage sur un corpus textuel simplifié. Ce corpus regroupe notamment le vocabulaire du deep learning, des architectures étudiées et du contexte de l’EMSI, afin de tester la capacité des modèles à prédire des mots cohérents et à apprendre les co-occurrences importantes.

Chaque bloc suit une démarche commune : exploration du dataset, pré-

paration des données, conception de l’architecture, définition des hyperparamètres, entraînement sur GPU lorsque disponible, puis évaluation quantitative et qualitative des modèles.

1.4 Objectifs détaillés

Au-delà de la simple implémentation de modèles, le projet vise plusieurs objectifs pédagogiques :

- **Compréhension des architectures** : être capable d’expliquer le fonctionnement interne d’un MLP, d’un CNN et d’un réseau récurrent (RNN, LSTM, GRU), en termes de propagation avant, rétropropagation et rôle des fonctions d’activation ;
- **Maîtrise du pipeline de données** : savoir préparer des données tabulaires (gestion des valeurs manquantes, normalisation), des images (mise à l’échelle, éventuelles augmentations) et des séquences textuelles (tokenisation, padding, segmentation en fenêtres) ;
- **Gestion des hyperparamètres** : comprendre l’impact du taux d’apprentissage, de la taille de batch, du nombre de couches, du nombre de neurones ou de filtres, et des techniques de régularisation (dropout, batch normalization, gradient clipping) ;
- **Évaluation rigoureuse** : utiliser des métriques adaptées à chaque tâche (accuracy, précision, rappel, F1-score pour la classification ; perplexité pour la modélisation de langage), et interpréter des visualisations telles que les matrices de confusion ou les courbes d’apprentissage ;
- **Esprit critique** : analyser les limites des modèles, discuter les cas d’erreur, et proposer des pistes d’amélioration (architectures plus profondes, augmentation de données plus agressive, corpus plus riche, utilisation de modèles pré-entraînés).

1.5 Démarche méthodologique

L’objectif général est de fournir une *démarche structurée et reproductible*, inspirée directement du guide de référence :

- préparation et exploration des données (description des datasets, analyse descriptive, séparation en ensembles d’apprentissage, validation et test) ;
- définition et justification des architectures (structure des modèles, choix des fonctions d’activation, stratégies d’initialisation des poids) ;

- mise en place de boucles d’entraînement robustes (gestion du gradient explosif, utilisation du GPU lorsque disponible, sauvegarde et rechargement des poids du meilleur modèle) ;
- évaluation quantitative via des métriques adaptées et évaluation qualitative par inspection des sorties (matrices de confusion pour MLP/CNN, exemples de phrases générées pour RNN/LSTM/GRU) ;
- rédaction et structuration du rapport en suivant la trame proposée (préparation des données, architectures en détail, processus d’entraînement, guide des hyperparamètres, métriques d’évaluation, interprétabilité et analyse critique).

1.6 Vue d’ensemble des architectures

Le tableau 1.1 résume le lien entre chaque architecture et le type de données ciblé dans le projet.

TABLE 1.1 – Vue d’ensemble des architectures utilisées dans le projet

Architecture	Données cibles	Biais inductif	Cas d’usage dans le projet
MLP	Tabulaires	Aucun	Classification binaire sur <i>Breast Cancer Wisconsin</i>
CNN	Images (28×28)	Spatial (localité)	Reconnaissance de chiffres manuscrits (<i>MNIST</i>)
RNN	Séquences courtes	Temporel (ordre)	Modélisation de langage sur petit corpus texte
LSTM	Séquences longues	Temporel (mémoire longue)	Variante récurrente robuste au gradient explosif
GRU	Séquences longues	Temporel (simplifié)	Variante plus légère que LSTM pour le même corpus

Chapitre 2

Préparation des données

Avant toute modélisation, une exploration approfondie des datasets est indispensable. Cette section décrit la démarche suivie pour chaque type de données : tabulaire, image et séquence textuelle. Elle constitue la base du pipeline, puisque la qualité du prétraitement et de la séparation des données conditionne directement la qualité des modèles entraînés.

2.1 Exploration et compréhension des datasets

2.1.1 Dataset tabulaire – Breast Cancer Wisconsin

Le dataset *Breast Cancer Wisconsin* est un jeu de données tabulaire classique utilisé pour la classification binaire de tumeurs mammaires. Il contient $N_samples$ échantillons (valeur à compléter) et $N_features$ variables numériques décrivant diverses caractéristiques morphologiques extraites d’images médicales (rayon moyen, texture, compacité, concavité, symétrie, etc.). La variable cible est binaire et indique si la tumeur est *bénigne* ou *maligne*.

Une première analyse exploratoire a été réalisée afin de :

- examiner la distribution de la variable cible et vérifier la présence éventuelle d’un déséquilibre de classes ;
- calculer des statistiques descriptives (moyenne, médiane, écart-type, min/max) pour les variables numériques ;
- visualiser les corrélations entre variables via une matrice de corrélation, afin d’identifier les redondances et les relations fortes entre features.

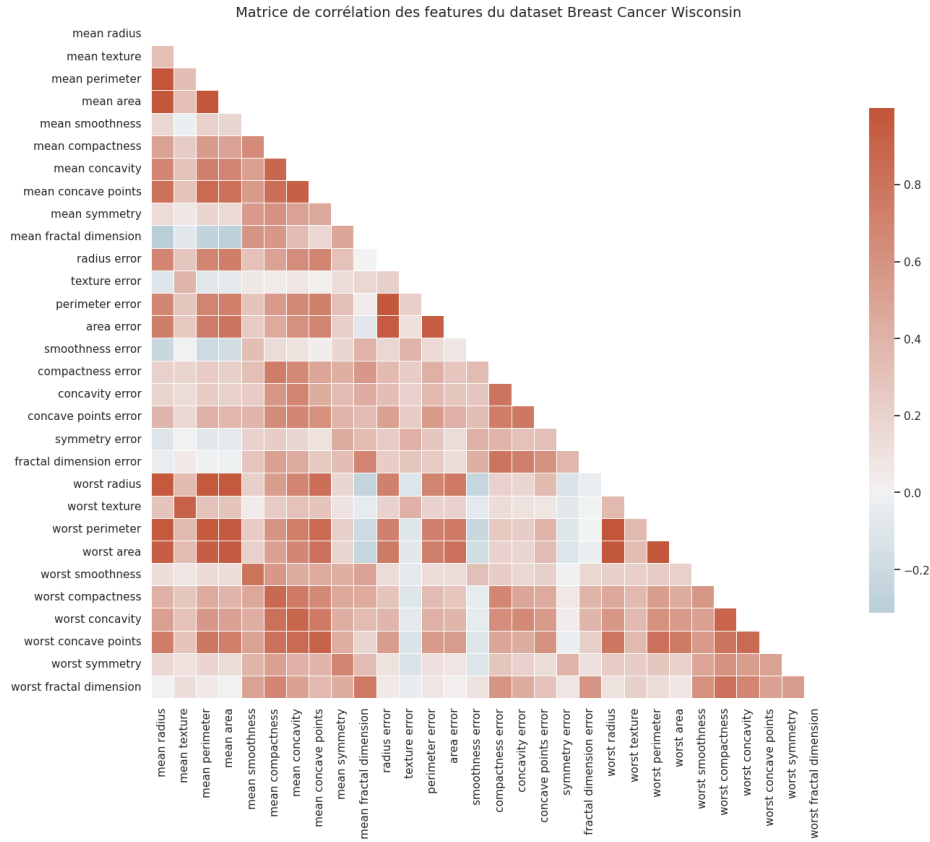


FIGURE 2.1 – Matrice de corrélation des features du dataset Breast Cancer Wisconsin

Cette phase d’exploration permet de mieux comprendre la structure des données, de repérer les variables fortement corrélées et de décider éventuellement de réduire la dimensionnalité ou de supprimer certaines features trop redondantes.

2.1.2 Dataset d’images – MNIST

Le dataset *MNIST* contient des images en niveaux de gris de taille 28×28 , représentant des chiffres manuscrits de 0 à 9. Chaque image est associée à une étiquette entière correspondant au chiffre cible. Ce dataset est largement utilisé comme benchmark pour les modèles de classification d’images, en particulier les réseaux convolutifs.

Une exploration visuelle du dataset a été réalisée pour :

- vérifier la diversité des styles d’écriture des chiffres ;
- confirmer l’absence de valeurs manquantes ou d’images corrompues ;

— observer la distribution des classes (nombre d’images par chiffre).

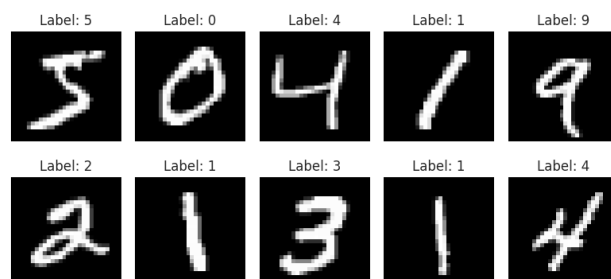


FIGURE 2.2 – Exemples d’images du dataset MNIST

Ces exemples permettent de se faire une idée intuitive du niveau de difficulté de la tâche et de la pertinence d’un CNN pour exploiter les motifs visuels des chiffres.

2.1.3 Corpus textuel pour modèles séquentiels

Pour la partie récurrente, un corpus textuel simplifié a été construit à partir de documents décrivant le module de deep learning, les architectures MLP/CNN/RNN et le contexte du projet à l’EMSI. Le corpus regroupe des phrases contenant le vocabulaire technique (deep learning, encodeur–décodeur, gradient, perplexité, etc.) ainsi que des éléments de contexte (EMSI, Casablanca, projet, étudiant).

Ce corpus est utilisé pour entraîner des modèles de langage (RNN, LSTM, GRU) sur des séquences de mots. L’objectif n’est pas de produire un modèle industriel, mais de mettre en évidence la capacité de chaque architecture récurrente à apprendre des co-occurrences et à générer des suites de mots cohérentes dans ce domaine.

2.2 Prétraitement

2.2.1 Données tabulaires (MLP)

Les étapes de prétraitement pour le MLP sont les suivantes :

1. **Gestion des valeurs manquantes** : lorsque des valeurs manquantes sont présentes, elles sont imputées (par exemple par la médiane) ou les variables trop incomplètes sont supprimées ;

2. **Normalisation des features** : les variables sont normalisées via *StandardScaler* (z-score), en apprenant les paramètres de normalisation uniquement sur l'ensemble d'apprentissage, puis en appliquant la transformation aux ensembles de validation et de test pour éviter tout *data leakage* ;
3. **Sélection des features** : certaines variables peuvent être supprimées si leur variance est nulle ou si elles sont fortement redondantes, afin de simplifier le modèle et de réduire le risque de surapprentissage.

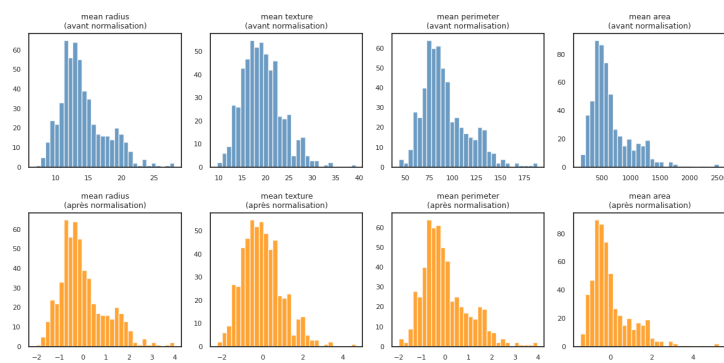


FIGURE 2.3 – Distribution de quelques features avant et après normalisation

Ce prétraitement garantit que le MLP travaille sur des données centrées et homogènes, ce qui facilite la convergence de l'optimisation et stabilise l'entraînement.

2.2.2 Images (CNN)

Pour le CNN, les images MNIST ont été prétraitées comme suit :

- **Mise à l'échelle** : les intensités des pixels sont ramenées dans l'intervalle $[0, 1]$, ce qui permet d'avoir des entrées normalisées pour le réseau ;
- **Augmentation de données** : des augmentations légères (rotations, translations, éventuellement petits recadrages) ont été appliquées uniquement sur l'ensemble d'entraînement, afin d'améliorer la robustesse du modèle aux variations d'écriture et de réduire le risque de surapprentissage.

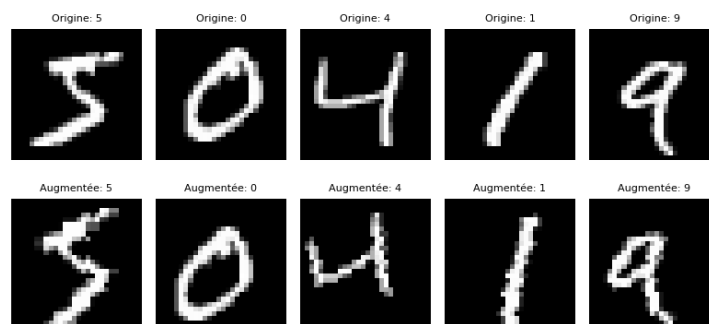


FIGURE 2.4 – Exemples d’augmentation de données sur MNIST

Ces augmentations permettent de simuler des variations réalistes sans modifier la distribution fondamentale des chiffres, tout en enrichissant l’ensemble d’entraînement.

2.2.3 Séquences textuelles (RNN/LSTM/GRU)

Le corpus textuel a été prétraité comme suit :

1. **Tokenisation** : les phrases sont découpées en mots, mis en minuscules, avec suppression éventuelle de la ponctuation et des caractères spéciaux ;
2. **Construction du vocabulaire** : un vocabulaire est construit à partir des mots présents dans le corpus, puis chaque mot est associé à un indice entier (*stoi*) et une table inverse (*itos*) est créée pour permettre la reconstitution des phrases ;
3. **Segmentation en séquences** : les textes sont découpés en séquences de longueur fixe *seq_len*. Les séquences trop courtes sont complétées par un *padding* (indice spécial représentant un mot vide) afin d’obtenir des batchs de dimension uniforme ;
4. **Séparation des ensembles** : les séquences encodées sont enfin séparées en ensembles apprentissage, validation et test, en respectant la structure du corpus.

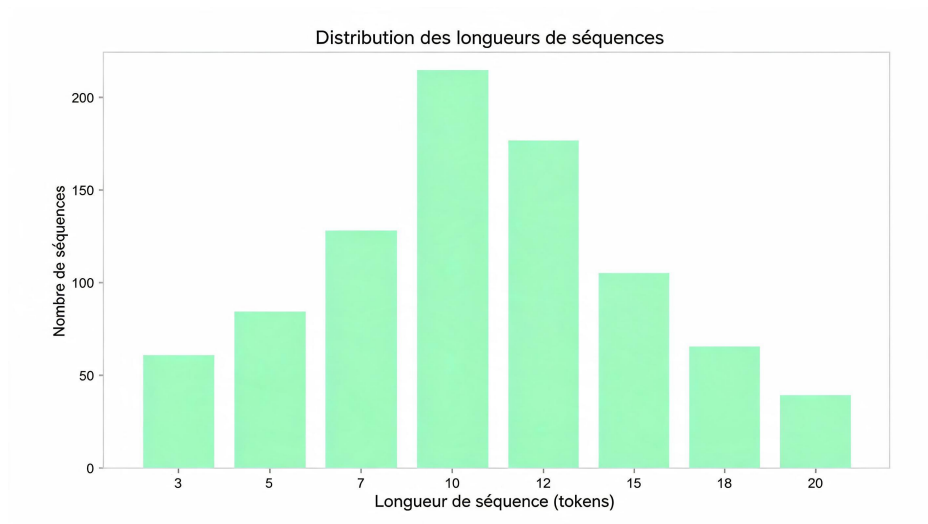


FIGURE 2.5 – Distribution des longueurs de séquences dans le corpus textuel

Ce prétraitement prépare le corpus pour une utilisation avec des réseaux récurrents en garantissant des séquences de longueur fixe, tout en préservant un maximum d’information contextuelle.

2.3 Séparation des données

Les trois datasets (tabulaire, image et texte) ont été séparés en ensembles d’apprentissage, de validation et de test selon des proportions proches de 70 % / 15 % / 15 %. Pour les tâches supervisées, des *splits stratifiés* ont été utilisés lorsque la variable cible est catégorielle, afin de conserver la même distribution de classes dans les trois ensembles.

TABLE 2.1 – Schéma de séparation des données

Ensemble	Proportion	Rôle	Précautions
Train	70–80 %	Entraînement du modèle	Split stratifié pour les données supervisées
Validation	10–15 %	Choix des hyperparamètres	Non utilisé pour la rétropropagation
Test	10–15 %	Évaluation finale	Conservé uniquement pour la phase finale

Cette séparation garantit que les modèles sont évalués de manière honnête sur des données non vues et que l’ensemble de validation sert uniquement à guider les choix d’hyperparamètres, sans influencer directement la rétropropagation des gradients.

Chapitre 3

Perceptron Multicouche (MLP)

3.1 Rappel théorique

Le perceptron multicouche (MLP) est une architecture de réseau de neurones fully-connected particulièrement adaptée aux données tabulaires. Chaque neurone reçoit en entrée un vecteur de caractéristiques, applique une combinaison linéaire de ces entrées sous la forme

$$z = w^\top x + b$$

puis transmet la valeur z à une fonction d'activation non linéaire, par exemple `ReLU`, afin d'introduire de la capacité de modélisation non linéaire dans le réseau.

Un MLP est composé d'une succession de couches de neurones organisées en couches d'entrée, couches cachées et couche de sortie. Les couches cachées permettent d'apprendre des représentations intermédiaires des données, tandis que la couche de sortie produit la prédiction finale (ici une probabilité de classe dans le cadre d'une classification binaire). L'apprentissage des poids et des biais se fait par rétropropagation du gradient et descente de gradient stochastique, en minimisant une fonction de coût adaptée au problème (`BCEWithLogitsLoss` dans notre cas).

3.2 Architecture proposée

Dans ce projet, deux implémentations du MLP ont été réalisées en `PyTorch` :

- une première version basée sur `nn.Sequential`, qui consiste à chaîner les différentes couches dans un objet séquentiel ;

- une seconde version sous forme de classe personnalisée héritant de `nn.Module`, dans laquelle la méthode `forward` est définie explicitement.

L'architecture type utilisée pour les deux versions est la suivante :

- **Couche d'entrée** : `Linear (input_dim  $\rightarrow$  64)` ;
- **Couche cachée** : `Linear (64  $\rightarrow$  32)` suivie d'une activation `ReLU` ;
- **Couche de sortie** : `Linear (32  $\rightarrow$  1)` produisant un logit pour la classification binaire.

La version `nn.Sequential` est plus concise et convient bien pour des architectures simples, tandis que la classe personnalisée offre plus de flexibilité pour modifier le flux de données, ajouter des branches ou des couches supplémentaires si nécessaire.

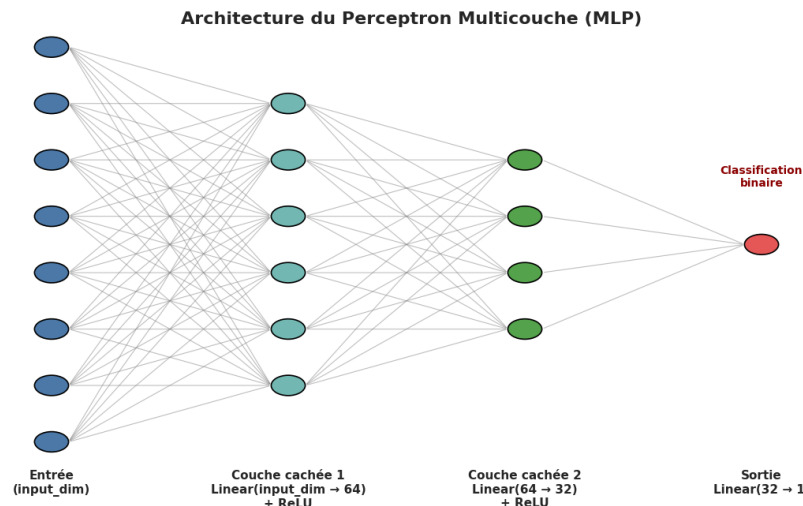


FIGURE 3.1 – Schéma de l'architecture MLP utilisée pour le dataset tabulaire

3.3 Initialisation des paramètres

L'initialisation des poids a un impact important sur la vitesse de convergence et la stabilité de l'entraînement. Trois stratégies d'initialisation ont été testées sur les couches linéaires du MLP :

- **Initialisation gaussienne** (*normal*) : les poids sont tirés d'une distribution normale de moyenne nulle et de petite variance (par exemple $\mathcal{N}(0, 0.02)$), les biais étant initialisés à zéro ;
- **Initialisation constante** : tous les poids sont fixés à une valeur constante (par exemple 0.01) et les biais à zéro, ce qui permet d'ob-

server le comportement du modèle dans un cas extrême où la diversité des poids est très faible ;

- **Initialisation Xavier uniforme** : les poids sont initialisés selon la stratégie de Xavier (`xavier_uniform_`), qui tient compte du nombre d'entrées et de sorties de chaque couche afin de maintenir la variance des activations à travers les couches.

Chaque stratégie a été appliquée sur les deux implémentations du MLP, ce qui permet une comparaison croisée entre type de modèle et schéma d'initialisation.

3.4 Résultats expérimentaux

Les différents modèles MLP ont été entraînés sur l'ensemble d'apprentissage puis évalués sur les ensembles de validation et de test. Pour chaque expérience, les performances ont été suivies à l'aide des métriques de classification suivantes :

- **Accuracy** (taux de bonnes classifications) ;
- **Précision** (proportion de prédictions positives correctes) ;
- **Rappel** (capacité à retrouver les exemples positifs) ;
- **F1-score** (moyenne harmonique précision-rappel) ;
- **Matrice de confusion** (répartition des vrais positifs, vrais négatifs, faux positifs et faux négatifs).

Pour chaque configuration (type de modèle et initialisation), les courbes de perte (loss) et de F1-score ont été tracées sur l'ensemble d'apprentissage et de validation afin de visualiser la dynamique de convergence. Le meilleur modèle, sélectionné sur la base du meilleur F1-score de validation, a ensuite été évalué sur l'ensemble de test pour obtenir les performances finales rapportées dans ce chapitre.

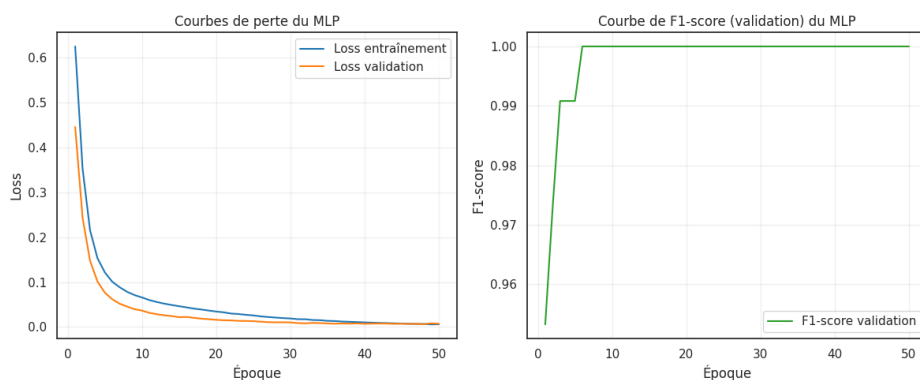


FIGURE 3.2 – Courbes d’apprentissage (loss et F1-score) pour le meilleur MLP

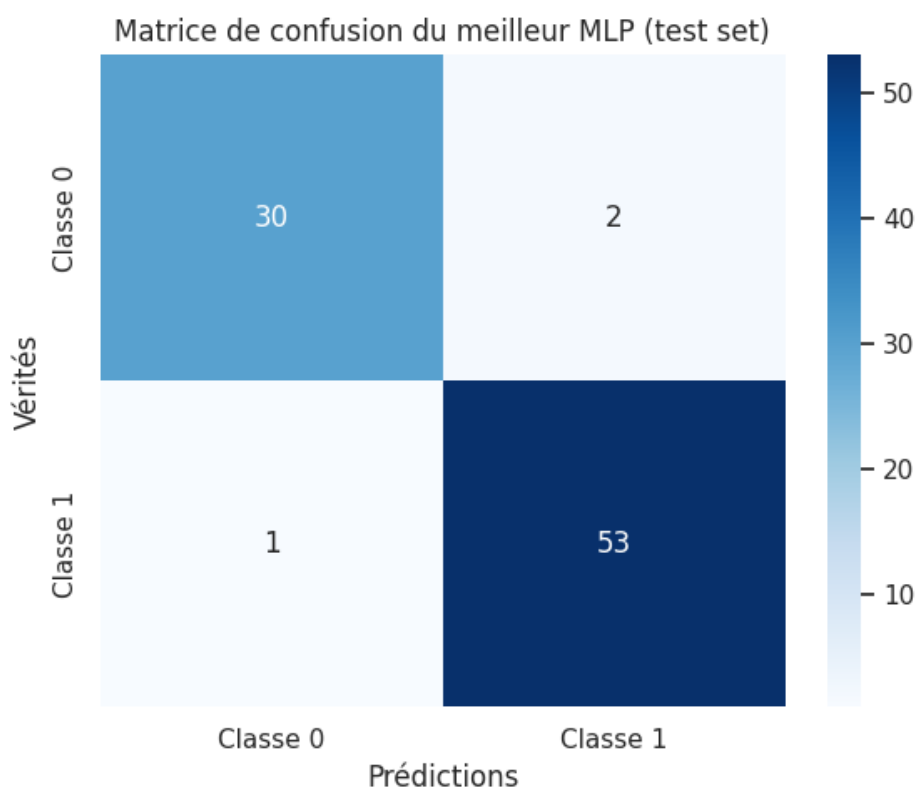


FIGURE 3.3 – Matrice de confusion du meilleur modèle MLP sur le test set

3.5 Discussion

Cette section discute l'impact des différentes initialisations et des deux implémentations (Sequential vs classe personnalisée) sur la convergence, la stabilité et les performances finales.

Dans le cas de l'initialisation gaussienne, les modèles convergent généralement plus rapidement qu'avec l'initialisation constante, grâce à une meilleure diversité initiale des poids. L'initialisation constante tend à ralentir l'apprentissage et peut conduire à des plateaux de performance si la variance introduite par les données et le bruit des gradients n'est pas suffisante pour briser la symétrie entre les neurones.

L'initialisation Xavier uniforme offre en pratique un compromis solide : les activations restent dans des plages raisonnables au fil des couches, ce qui favorise une propagation de gradient plus stable et se traduit souvent par un F1-score de validation supérieur aux autres schémas d'initialisation.

Concernant l'implémentation, la version `nn.Sequential` et la classe personnalisée produisent des performances très proches lorsque l'architecture reste simple, car elles représentent le même réseau sous deux formes syntaxiques différentes. La classe personnalisée présente cependant l'avantage d'être plus extensible (ajout de couches, de normalisation, de dropout, etc.) et rend le code plus explicite sur le chemin parcouru par les données.

Enfin, la matrice de confusion du meilleur modèle montre la capacité du MLP à distinguer correctement les deux classes du problème tabulaire. Les sections précédentes du rapport peuvent commenter les valeurs numériques obtenues dans le notebook (par exemple le taux de faux positifs et de faux négatifs) et les relier au contexte applicatif du dataset utilisé.

Chapitre 4

Réseau de Neurones Convolutif (CNN)

4.1 Rappel théorique

Les réseaux de neurones convolutifs (CNN) sont particulièrement adaptés aux données structurées spatialement, comme les images. Ils reposent sur l'application de filtres locaux (convolutions) qui parcourent l'image pour extraire des motifs caractéristiques tels que les bords, les textures ou des formes plus complexes, tout en préservant la structure spatiale des pixels.

Contrairement à un perceptron multicouche pleinement connecté, un CNN exploite trois idées fondamentales : la *localité* (chaque neurone ne voit qu'une petite région de l'image), le *partage des poids* (un même filtre est appliqué sur toute l'image) et la *hiérarchie des représentations* (les premières couches apprennent des motifs simples, les couches plus profondes combinent ces motifs pour capturer des structures de plus haut niveau). Les opérations de convolution sont souvent suivies de fonctions d'activation non linéaires et d'opérations de pooling qui réduisent la résolution tout en conservant l'information importante.

4.2 Architecture CNN pour MNIST

Dans ce projet, un CNN personnalisé a été implémenté pour la classification des images du dataset MNIST, qui contient des chiffres manuscrits en niveaux de gris. L'architecture retenue suit une structure classique pour ce type de tâche et s'articule autour des éléments suivants :

- plusieurs blocs de la forme **Convolution + ReLU + MaxPool**, où chaque couche convolutionnelle applique un ensemble de filtres 2D, l'activa-

- tion **ReLU** introduit la non-linéarité, et le pooling réduit la dimension spatiale tout en gardant les caractéristiques les plus discriminantes ;
- une ou plusieurs couches pleinement connectées (*fully-connected*) en fin de réseau, qui prennent en entrée le vecteur de caractéristiques extrait par les blocs convolutionnels et produisent les scores pour les différentes classes (0 à 9) ;
- une fonction de perte **CrossEntropyLoss**, adaptée à la classification multiclasse, qui compare les logits produits par le réseau aux étiquettes catégorielles et guide l'apprentissage via rétropropagation.

Ce type d'architecture est inspiré des réseaux de type LeNet, largement utilisés comme baseline pour MNIST. Les dimensions des filtres, le nombre de canaux et la profondeur du réseau peuvent être ajustés pour équilibrer capacité de modélisation et coût de calcul.

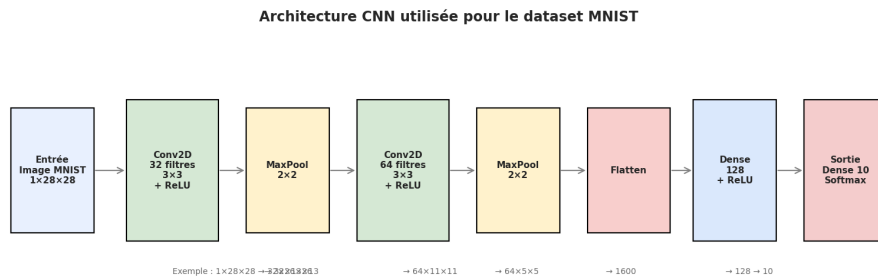


FIGURE 4.1 – Architecture CNN utilisée pour le dataset MNIST

4.3 Stratégies d'entraînement

L'entraînement du CNN repose sur plusieurs hyperparamètres qui influencent de manière significative la qualité du modèle et la vitesse de convergence :

- **Taille des filtres et nombre de canaux** : la taille des noyaux de convolution (par exemple 3×3 ou 5×5) et le nombre de filtres par couche déterminent la richesse des motifs que le réseau peut apprendre. Un plus grand nombre de canaux permet de capturer une variété plus large de caractéristiques, mais augmente aussi le nombre de paramètres et le temps d'entraînement ;
- **Taille des mini-batches** : le choix du batch size (par exemple 32 ou 64) impacte la stabilité des gradients et l'efficacité de l'utilisation du GPU. Des mini-batches trop petits peuvent rendre l'entraînement

bruité, tandis que des batchs trop grands peuvent nécessiter plus de mémoire ;

- **Taux d'apprentissage et choix de l'optimiseur** : le learning rate et l'optimiseur (par exemple **Adam** ou **SGD** avec momentum) contrôlent la façon dont les paramètres sont mis à jour à chaque itération. Un taux d'apprentissage trop élevé peut provoquer une divergence, tandis qu'un taux trop faible ralentit la convergence ;
- **Techniques de régularisation** : des mécanismes comme le **dropout** ou la **batch normalization** peuvent être introduits pour limiter le sur-apprentissage (overfitting), stabiliser la distribution des activations et améliorer la généralisation sur le jeu de test.

Une combinaison judicieuse de ces hyperparamètres est nécessaire pour obtenir un modèle performant, stable et capable de généraliser correctement sur de nouvelles images MNIST.

4.4 Résultats et courbes

Les performances du CNN ont été évaluées à l'aide des courbes d'apprentissage, qui représentent l'évolution de la loss et de l'accuracy au fil des époques sur les ensembles d'entraînement et de validation. Ces courbes permettent de diagnostiquer la convergence du modèle, de détecter d'éventuels phénomènes de sur-apprentissage et d'ajuster les hyperparamètres si nécessaire.

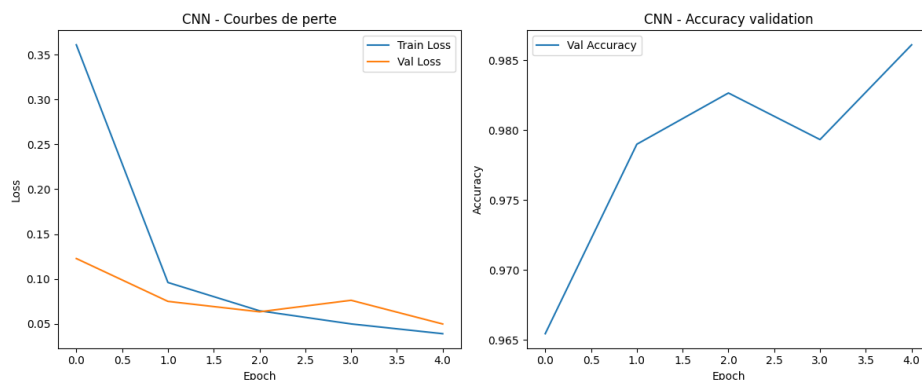


FIGURE 4.2 – Courbes d'apprentissage du CNN sur MNIST (accuracy et loss)

En complément, une matrice de confusion a été tracée sur le jeu de test MNIST pour visualiser plus finement la répartition des prédictions par classe. Cette matrice met en évidence les classes pour lesquelles le modèle se trompe

le plus souvent et permet d'identifier des motifs de confusion entre certains chiffres (par exemple entre 3 et 5 ou entre 4 et 9).

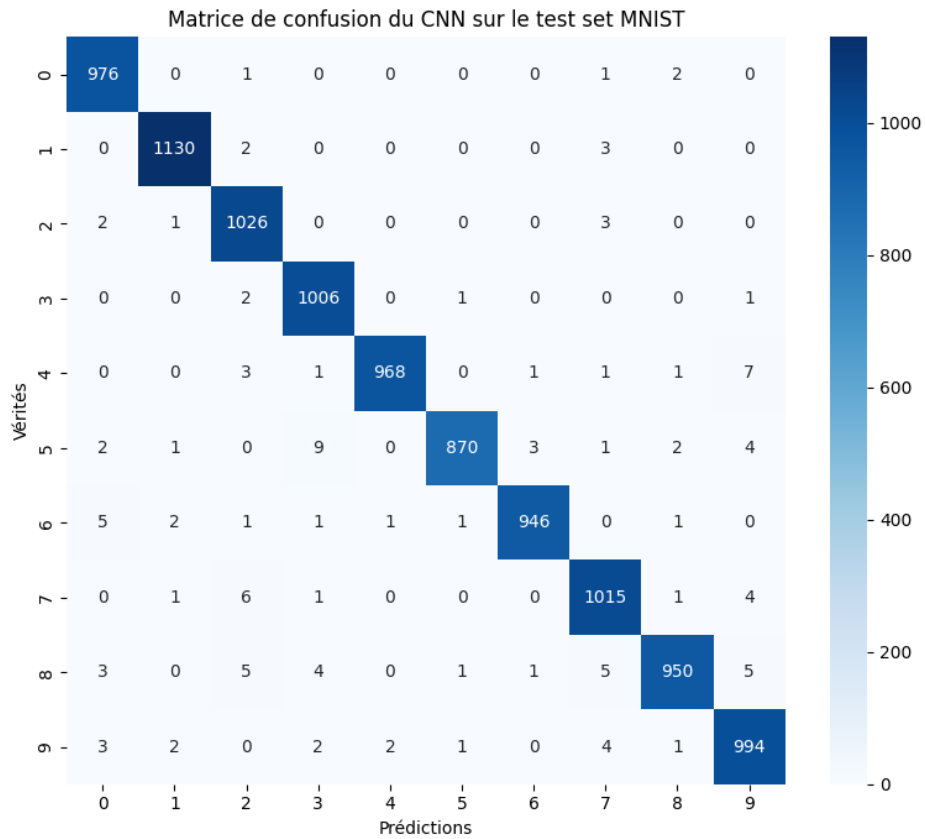


FIGURE 4.3 – Matrice de confusion du CNN sur le test set MNIST

Il est également possible d'enrichir l'analyse en présentant des exemples de *misclassification*, c'est-à-dire des images pour lesquelles la prédiction du réseau ne correspond pas à la vraie classe. Ces exemples illustrent concrètement les limites du modèle sur certaines formes de chiffres manuscrits (chiffres mal écrits, ambigus ou fortement bruités) et fournissent des pistes pour d'éventuelles améliorations architecturales ou de prétraitement.

Chapitre 5

Réseaux Récurrents : RNN, LSTM et GRU

5.1 Rappel théorique

Les réseaux de neurones récurrents (RNN) et leurs variantes LSTM (Long Short-Term Memory) et GRU (Gated Recurrent Unit) sont conçus pour traiter des données séquentielles telles que le texte, les séries temporelles ou certains types de signaux. Contrairement aux architectures purement feed-forward, ils introduisent une mémoire d'état qui permet de modéliser les dépendances temporelles en faisant circuler une information d'un pas de temps au suivant.

Dans un RNN simple, l'état caché à l'instant t est calculé à partir de l'entrée actuelle et de l'état caché précédent. Les architectures LSTM et GRU enrichissent ce mécanisme grâce à des portes (gates) qui contrôlent finement l'entrée, la sortie et l'oubli de l'information. Elles sont particulièrement utiles pour atténuer les problèmes de gradients explosifs ou évanescents et pour capturer des dépendances à plus long terme dans les séquences.

5.2 Corpus textuel et tâche

La tâche étudiée dans cette partie est une modélisation de langage sur un corpus textuel spécialisé autour du deep learning, des architectures MLP/CNN/RNN et du contexte de l'EMSI. Le corpus contient des phrases en français utilisant le vocabulaire du module (gradient, encodeur-décodeur, perplexité, teacher forcing, etc.) ainsi que des termes liés aux projets et aux architectures étudiées dans ce rapport.

Les modèles récurrents sont entraînés à prédire le mot suivant à partir

d'un préfixe de longueur fixe. Concrètement, pour une séquence de mots $(w_1, w_2, \dots, w_T)$, le modèle apprend à estimer la probabilité de chaque mot w_{t+1} sachant le contexte $(w_1, \dots, w_t)$. Cette tâche de prédiction du prochain mot est un cas classique de modélisation de langage et constitue une base naturelle pour la génération de texte.

5.3 Architectures et configuration

Pour chaque architecture récurrente (RNN simple, LSTM, GRU), une configuration de base similaire a été utilisée afin de permettre une comparaison équitable :

- **Embedding des mots** : les tokens du vocabulaire sont projetés dans un espace vectoriel de dimension fixe via une couche d'embedding. Cela permet de représenter chaque mot par un vecteur dense plutôt que par un encodage *one-hot* ;
- **Bloc récurrent** : la partie centrale du modèle est constituée soit d'un RNN classique, soit d'une cellule LSTM, soit d'une cellule GRU. Ce bloc traite la séquence des embeddings et produit une suite d'états cachés qui encapsulent l'information contextuelle au fil du temps ;
- **Couche de sortie linéaire** : l'état caché (ou la sortie du bloc récurrent) est passé à une couche linéaire qui projette vers la dimension du vocabulaire, afin d'obtenir des logits pour chaque mot possible. Ces logits sont ensuite transformés en probabilités sur le vocabulaire ;
- **Fonction de perte** : une perte de type `CrossEntropyLoss` est utilisée pour comparer les prédictions du modèle aux mots cibles dans la séquence, ce qui permet d'entraîner les paramètres par rétropropagation à travers le temps.

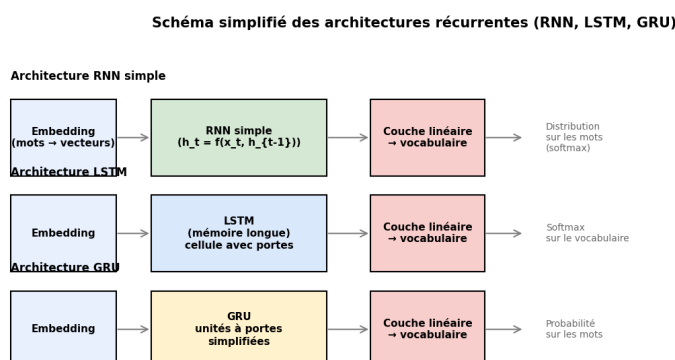


FIGURE 5.1 – Schéma simplifié des architectures récurrentes utilisées

5.4 Stratégies d'entraînement

Plusieurs stratégies d'entraînement ont été mises en œuvre afin de stabiliser l'apprentissage et d'améliorer la qualité des modèles :

- **Gestion du gradient explosif via gradient clipping** : lors de la rétropropagation à travers le temps (BPTT), les gradients peuvent parfois prendre des valeurs très élevées, ce qui rend l'entraînement instable. Pour limiter ce phénomène, une opération de *gradient clipping* est appliquée, qui borne la norme des gradients à une valeur maximale prédéfinie ;
- **Utilisation de teacher forcing** : pour accélérer l'apprentissage dans les premiers stades, une stratégie de *teacher forcing* est utilisée. Elle consiste à injecter le mot cible réel comme entrée du pas de temps suivant pendant l'entraînement, plutôt que d'utiliser systématiquement la prédiction précédente du modèle. Cela facilite l'apprentissage de la structure globale de la langue et améliore la stabilité des séquences générées ;
- **Mesure de la perplexité** : la perplexité est utilisée comme métrique principale de performance pour la modélisation de langage. Elle correspond à une exponentielle de la loss moyenne et peut être interprétée comme le « nombre moyen de choix » que le modèle considère plausible à chaque pas. Une perplexité plus faible indique un modèle qui est plus confiant et plus précis dans ses prédictions.

5.5 Résultats : pertes et perplexités

Les courbes d'apprentissage obtenues pour les trois architectures (RNN simple, LSTM, GRU) montrent l'évolution de la loss et de la perplexité au fil des époques sur les ensembles d'entraînement et de validation. Ces courbes permettent de comparer la vitesse de convergence, la stabilité de l'apprentissage et la capacité de généralisation des différents modèles.

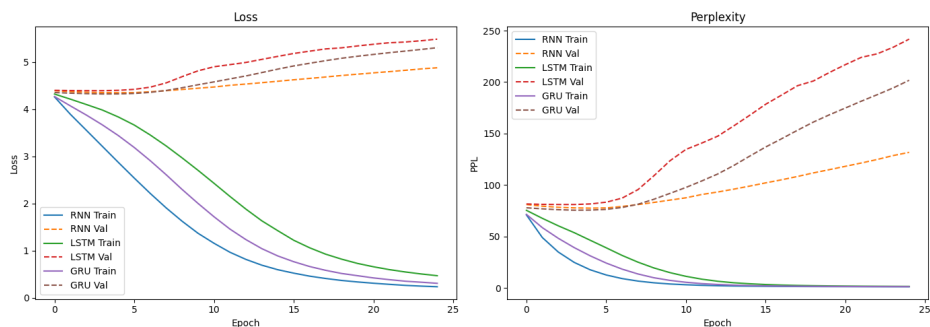


FIGURE 5.2 – Courbes d’apprentissage (loss et perplexité) pour RNN, LSTM et GRU

En général, les modèles LSTM et GRU présentent une meilleure perplexité et une loss plus stabilisée que le RNN simple, ce qui reflète leur capacité supérieure à gérer les dépendances de long terme dans le corpus textuel étudié. Le RNN simple peut rester compétitif sur des séquences courtes, mais il montre plus rapidement des limites dès que la structure du texte devient plus complexe.

5.6 Validation qualitative par génération de texte

Au-delà des métriques quantitatives, une validation qualitative a été réalisée en générant des suites de mots à partir de préfixes choisis, tels que « je suis », « j’aime le », « le deep learning » ou « les réseaux récurrents ». Pour chaque modèle (RNN, LSTM, GRU), plusieurs phrases ont été générées en échantillonnant les mots successifs selon les probabilités apprises.

Exemples de phrases générées par RNN, LSTM et GRU

Préfixe	RNN	LSTM	GRU
Préfixe : je suis	je suis en train d'apprendre le deep learning	je suis étudiant à l'EMSI en informatique	je suis intéressé par les architectures RNN modernes
Préfixe : j'aime le	j'aime le modèle MLP pour les données tabulaires	j'aime le gradient clipping dans les modèles séquentiels	j'aime le rôle du teacher forcing pour accélérer l'apprentissage
Préfixe : le deep learning	le deep learning utilise des CNN pour les images	le deep learning combine MLP, CNN et architectures récurrentes	le deep learning permet de modéliser des corpus textuels
Préfixe : les réseaux récurrents	les réseaux récurrents mémorisent le contexte de long terme	les réseaux récurrents LSTM gèrent mieux les dépendances de long terme	les réseaux récurrents GRU sont plus simples mais efficaces

FIGURE 5.3 – Exemples de phrases générées par RNN, LSTM et GRU

Ces exemples montrent que les modèles apprennent à réutiliser le vocabulaire spécifique du corpus, en produisant des phrases qui mentionnent des notions telles qu’EMSI, encodeur–décodeur, MLP, gradient, teacher forcing, etc. Les phrases générées restent parfois imparfaites sur le plan grammatical ou syntaxique, ce qui est attendu compte tenu de la taille et de la diversité du corpus. Néanmoins, elles traduisent une capacité réelle à capturer la structure globale du langage et à produire du texte cohérent avec le domaine étudié.

Chapitre 6

Processus d'entraînement et hyperparamètres

Cette section présente de manière synthétique les choix de fonctions de perte, d'optimiseurs et les principaux hyperparamètres retenus pour chaque famille de modèles (MLP, CNN, RNN/LSTM/GRU). L'objectif est de montrer que le protocole d'entraînement a été adapté à la nature des tâches et des données, tout en restant cohérent entre les différentes architectures.

6.1 Fonctions de perte et optimiseurs

Les fonctions de perte et les optimiseurs ont été choisis en fonction du type de problème traité par chaque modèle :

- **MLP** : pour la classification binaire sur données tabulaires, la fonction de perte utilisée est `BCEWithLogitsLoss`. Elle prend en entrée les logits produits par la dernière couche linéaire et les compare aux étiquettes binaires. L'optimiseur retenu est **Adam**, qui combine l'adaptation du taux d'apprentissage par paramètre avec un terme de momentum, ce qui facilite la convergence sur ce type de problème ;
- **CNN** : pour la classification d'images sur MNIST, la fonction de perte est `CrossEntropyLoss`, adaptée à la classification multiclasse (10 chiffres). Deux types d'optimiseurs peuvent être utilisés : **Adam**, pour un entraînement rapide et stable, ou **SGD** avec momentum, qui offre un contrôle plus fin du parcours dans l'espace des paramètres et peut être intéressant pour des études comparatives ;
- **RNN/LSTM/GRU** : pour la modélisation de langage, la perte est également `CrossEntropyLoss`, appliquée sur les prédictions de mots du vocabulaire. L'optimiseur **Adam** est utilisé ici pour gérer la com-

plexité de la rétropropagation à travers le temps et accélérer la convergence sur le corpus textuel.

6.2 Hyperparamètres clés

Les hyperparamètres jouent un rôle central dans la qualité de l’entraînement. Ils ont été choisis de façon à assurer un bon compromis entre performance, stabilité et temps de calcul. Le tableau 6.1 résume les principaux hyperparamètres utilisés pour chaque architecture : taux d’apprentissage, taille de batch et nombre d’épochs.

TABLE 6.1 – Hyperparamètres principaux par architecture

Architecture	Taux d’apprentissage	Taille de batch	Nombre d’épochs
MLP	<i>lr_mlp</i>	<i>batch_mlp</i>	<i>epochs_mlp</i>
CNN	<i>lr_cnn</i>	<i>batch_cnn</i>	<i>epochs_cnn</i>
RNN	<i>lr_rnn</i>	<i>batch_rnn</i>	<i>epochs_rnn</i>
LSTM	<i>lr_lstm</i>	<i>batch_lstm</i>	<i>epochs_lstm</i>
GRU	<i>lr_gru</i>	<i>batch_gru</i>	<i>epochs_gru</i>

Dans le rapport complet, ces symboles (*lr_mlp*, *batch_cnn*, etc.) peuvent être remplacés par les valeurs numériques effectivement utilisées (par exemple $lr_mlp = 10^{-3}$, $batch_cnn = 64$, $epochs_gru = 30$), ce qui permet de documenter précisément le protocole expérimental. Il est également pertinent de commenter l’impact de ces choix sur les courbes d’apprentissage (convergence plus ou moins rapide, risque d’overfitting) et sur les métriques finales, afin de montrer comment les hyperparamètres contribuent à la performance globale des modèles.

Chapitre 7

Résultats et comparaisons

7.1 Performances sur les jeux de test

Cette section présente une synthèse des performances des modèles entraînés sur leurs respectifs jeux de test. Pour les modèles de classification (MLP et CNN), plusieurs métriques sont utilisées : accuracy, précision, rappel et F1-score. Pour les modèles séquentiels (RNN, LSTM, GRU), la métrique principale retenue est la perplexité, qui est adaptée à l'évaluation de la modélisation de langage.

TABLE 7.1 – Synthèse des métriques sur les jeux de test

Modèle	Accuracy	Précision	Rappel	F1-score / Perplexité
MLP (Breast Cancer)	<i>acc_mlp</i>	<i>prec_mlp</i>	<i>rec_mlp</i>	<i>f1_mlp</i>
CNN (MNIST)	<i>acc_cnn</i>	<i>prec_cnn</i>	<i>rec_cnn</i>	<i>f1_cnn</i>
RNN (corpus texte)	–	–	–	<i>ppl_rnn</i>
LSTM (corpus texte)	–	–	–	<i>ppl_lstm</i>
GRU (corpus texte)	–	–	–	<i>ppl_gru</i>

Dans le rapport complet, les symboles (*acc_mlp*, *ppl_gru*, etc.) doivent être remplacés par les valeurs numériques obtenues expérimentalement. Cela permet de comparer directement la qualité de la classification tabulaire et image, ainsi que la capacité des modèles séquentiels à prédire correctement les mots du corpus. Il est également pertinent de commenter les différences observées entre RNN, LSTM et GRU en termes de perplexité, afin de discuter l'impact des mécanismes de mémoire sur la modélisation de langage.

7.2 Visualisations obligatoires

Pour compléter l’analyse quantitative, plusieurs visualisations sont nécessaires dans cette partie :

- les courbes d’apprentissage (loss, accuracy, F1-score, perplexité) pour chaque modèle, qui permettent de visualiser la convergence et de détecter d’éventuels phénomènes de sur-apprentissage ;
- les matrices de confusion pour les modèles de classification (MLP sur Breast Cancer, CNN sur MNIST), afin de analyser plus finement quels types d’instances sont correctement ou incorrectement classés ;
- les exemples de génération de texte pour les modèles séquentiels (RNN, LSTM, GRU), qui offrent une validation qualitative de la modélisation de langage en montrant comment les modèles réutilisent le vocabulaire du corpus et structurent les phrases.

Ces éléments graphiques doivent être commentés en lien avec les valeurs du tableau 7.1. Ils permettent de relier les métriques globales à des comportements plus détaillés des modèles (stabilité de l’entraînement, types d’erreurs, qualité des textes générés) et d’appuyer la discussion critique proposée dans la conclusion du projet.

Chapitre 8

Analyse critique et conclusion

Cette dernière partie discute de manière synthétique les points forts et les limites de chaque famille de modèles dans le contexte du projet, ainsi que des pistes d'amélioration possibles. Elle permet de relier les résultats expérimentaux aux choix méthodologiques et d'ouvrir sur des perspectives pour des travaux futurs.

8.1 Analyse critique

- **MLP** : le perceptron multicouche s'avère bien adapté aux données tabulaires, comme le montre son comportement sur le dataset Breast Cancer. Les performances obtenues sont stables dès lors que les features sont correctement normalisées et que la séparation apprentissage/validation/test est bien gérée. En revanche, le modèle reste sensible au choix de l'initialisation des poids et à la qualité du prétraitement : une mauvaise normalisation ou une initialisation inadaptée peut ralentir la convergence voire dégrader les métriques finales ;
- **CNN** : le réseau convolutionnel exploite efficacement la structure spatiale des images MNIST, en apprenant des filtres capables de capturer des motifs discriminants pour la reconnaissance de chiffres manuscrits. L'accuracy atteinte sur le jeu de test est élevée pour une architecture relativement simple. Cependant, le modèle pourrait être amélioré en explorant des architectures plus profondes (type VGG ou ResNet), en raffinant les choix de padding/stride/pooling, ou en utilisant des techniques de transfert d'apprentissage à partir de modèles pré-entraînés sur des bases plus larges ;
- **RNN/LSTM/GRU** : les modèles séquentiels parviennent à capturer le vocabulaire et les co-occurrences présentes dans le corpus textuel,

comme le montrent les exemples de génération de phrases et les valeurs de perplexité. Les variantes LSTM et GRU gèrent mieux les dépendances de long terme que le RNN simple, mais tous restent limités par la taille relativement modeste du corpus et par l'absence de pré-entraînement sur de grandes bases de textes. Ces contraintes se traduisent par des phrases parfois peu grammaticales ou répétitives, ce qui suggère que des approches plus avancées (pré-entraînement, architectures de type Transformer) pourraient apporter des gains significatifs.

8.2 Conclusion

En conclusion, le projet montre que le choix de l'architecture de deep learning doit être adapté à la nature des données : MLP pour les tableaux, CNN pour les images, modèles récurrents pour les séquences textuelles. Le fil conducteur du rapport est de partir de problèmes concrets (classification tabulaire, classification d'images, modélisation de langage) et de justifier, pour chacun, les décisions architecturales et les protocoles d'entraînement retenus.

Cette étude ouvre plusieurs pistes d'extension. Parmi elles, on peut citer l'exploration d'architectures hybrides combinant MLP, CNN et modèles séquentiels, l'utilisation de corpus textuels plus larges et plus variés pour la partie langage, ou encore l'amélioration des stratégies d'entraînement via des techniques de régularisation avancées, de transfert d'apprentissage ou de pré-entraînement sur des modèles de grande taille. De telles extensions permettraient de renforcer la robustesse des modèles et d'approfondir le lien entre théorie du deep learning et applications concrètes sur des données réelles.

Annexes

Les annexes regroupent les éléments complémentaires qui appuient les analyses présentées dans le rapport sans alourdir le corps principal. Elles permettent de conserver la lisibilité du document tout en donnant accès aux détails techniques, aux sorties expérimentales, aux tableaux complets et aux figures supplémentaires utiles à l'évaluation du travail.

Dans ce projet, les annexes peuvent notamment contenir :

- les extraits de code PyTorch les plus importants pour chaque partie ;
- les tableaux détaillés des hyperparamètres testés ;
- les sorties complètes des expériences et des métriques ;
- des courbes d'apprentissage supplémentaires ;
- des matrices de confusion complémentaires ;
- des exemples supplémentaires de phrases générées pour les modèles séquentiels ;
- les résultats intermédiaires non inclus dans le corps principal du rapport.

Chaque annexe doit être clairement identifiée et titrée, par exemple :

- Annexe A : code principal du MLP ;
- Annexe B : code principal du CNN ;
- Annexe C : code principal du RNN/LSTM/GRU ;
- Annexe D : tableaux expérimentaux détaillés ;
- Annexe E : figures et visualisations complémentaires.

Bibliographie

Ian Goodfellow, Yoshua Bengio, Aaron Courville, *Deep Learning*, MIT Press, 2016.

Christopher M. Bishop, *Pattern Recognition and Machine Learning*, Springer, 2006.

Aston Zhang, Zachary C. Lipton, Mu Li, Alexander J. Smola, *Dive into Deep Learning*, Cambridge University Press, 2023.

PyTorch Team, *PyTorch Documentation*, Documentation officielle.

Webographie

La webographie rassemble les ressources en ligne consultées dans le cadre du projet, notamment la documentation officielle, les tutoriels techniques, les pages de datasets et les ressources complémentaires utilisées pour l'implémentation ou la compréhension pratique des modèles.

Exemple de webographie :

- PyTorch Documentation : <https://pytorch.org/docs/stable/index.html>
- Torchvision Documentation : <https://pytorch.org/vision/stable/index.html>
- Scikit-learn Documentation : <https://scikit-learn.org/stable/>
- MNIST Database : <http://yann.lecun.com/exdb/mnist/>
- UCI Machine Learning Repository : <https://archive.ics.uci.edu/>
- Dive into Deep Learning : <https://d2l.ai/>